

Task-Adaptive Grounded 3D-Programmers Using 2D VLMs

Arman Raayatsanati^{1,*} Sombit Dey^{2,*} Anna-Maria Halacheva^{2,*} Jan-Nico Zaech²
 Luc Van Gool² Danda Pani Paudel²
¹TextQL ²INSAIT, Sofia University "St. Kliment Ohridski"

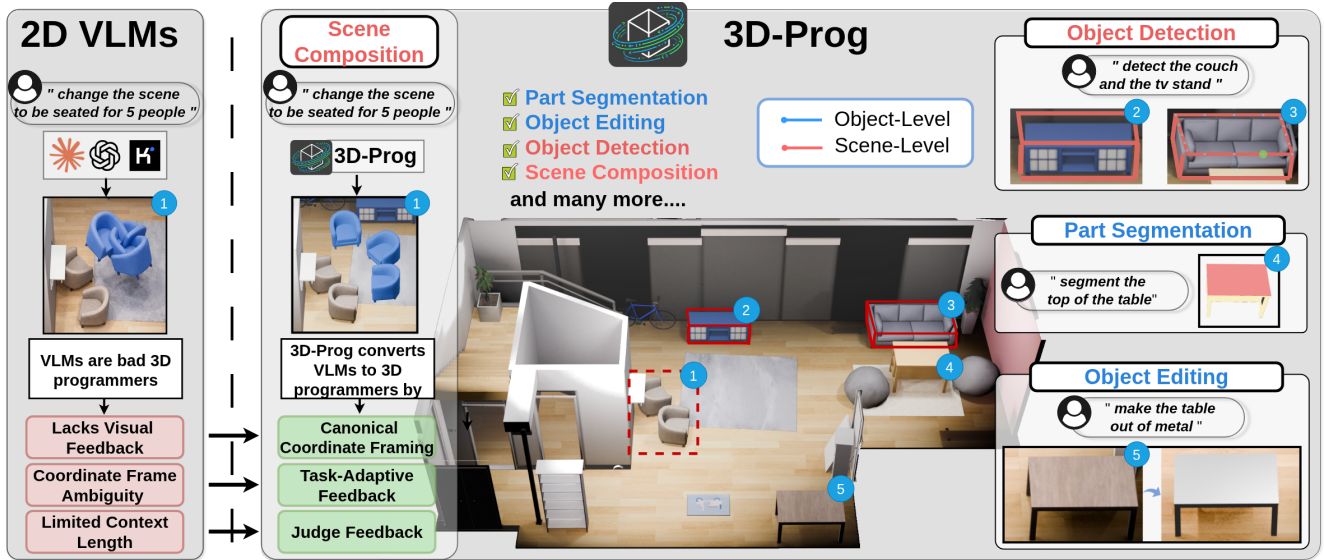


Figure 1. Today, 2D VLMs lack the zero-shot capacity to generate, modify, or reason about 3D assets. To bridge this gap, we introduce **3D-Prog**, a modular and flexible framework for 3D visual programming. By leveraging the semantic reasoning of 2D VLMs, 3D-Prog acts as a *3D programmer* that interprets multi-view 2D visualizations of a 3D asset within a canonical frame. The **3D-Prog** approach addresses a broad range of 3D tasks spanning generation, understanding, and compositional reasoning.

Abstract

Recent vision-language models (VLMs) exhibit remarkable generalization and reasoning abilities, yet 3D understanding in these models is limited by data scale, training diversity, and reasoning capacity. Instead of naively extending these models into 3D, we take a different approach: we enable powerful 2D VLMs to operate reliably in 3D by introducing 3D grounding and iterative feedback loops with two novel concepts: Canonical Coordinate Framing (CCF) and Task-Adaptive Feedback (TAF). CCF serves as a unified visual representation that anchors both inputs and outputs to a shared Euclidean coordinate system, solving common challenges in 3D grounding such as axis ambiguity, inconsistent metric scale, and floating references. Complementary to this structured framing of the 3D inputs, TAF closes

the reasoning loop with task-adaptive dynamic feedback that enables 2D VLMs to perform varied open-vocabulary tasks within their native visual context.

Building on this foundation, we introduce 3D-Prog, a 3D understanding, reasoning, and generation framework that jointly employs the capabilities of CCF and TAF together with powerful VLMs. Without requiring any retraining, 3D-Prog performs open-vocabulary 3D understanding, manipulation, and generation across both object-level and scene-level tasks. Our experiments show that the joint use of CCF and TAF transforms 2D VLMs into geometry-aware 3D programmers, achieving consistent, interpretable, and high-quality results across diverse 3D tasks.

1. Introduction

The pursuit of general-purpose 3D understanding and manipulation systems has traditionally relied on specialized

*Equal contribution.

[†]Research done as part of an ETH Zurich MSc thesis hosted by INSAIT.

models, each tailored for a specific geometric task [26, 33, 55]. Whether converting scanned objects into simulation-ready models, segmenting parts, or composing scenes from language, these systems depend on extensive 3D annotations, domain expertise, and task-specific training [17, 24, 42, 43]. This specialization fragments the 3D ecosystem and limits scalability to diverse real-world 3D tasks in robotics, graphics, and vision.

Meanwhile, vision-language models (VLMs) trained on massive 2D image-text corpora have demonstrated remarkable generalization, compositional reasoning, and zero-shot learning abilities [11, 14, 47]. Yet, despite their semantic power, these models operate within the 2D image manifold and lack explicit grounding in metric 3D space [51]. Emerging 3D VLMs attempt to bridge this gap, but their generalization remains constrained by limited data scale and diversity [25, 56]. As a result, neither approach provides sufficient performance; directly prompting 2D VLMs for 3D quantities leads to unstable and inconsistent predictions [51], while 3D VLMs lack a diversity of capabilities [56].

We address this challenge by introducing a task-adaptive and grounded 3D programming system built on two novel VLM communication mechanisms: **Canonical Coordinate Framing** (CCF) for 3D grounding and **Task-Adaptive Feedback** (TAF). CCF anchors both inputs and outputs to a shared Euclidean coordinate system by rendering multi-view orthographic projections with explicit, visible axes, metric scale marks, and a fixed global origin. These visual cues eliminate what we call coordinate-frame ambiguity and allow 2D VLMs to perform spatial reasoning within a consistent geometric frame. While spatial grounding is necessary, it is often insufficient for complex tasks. To address this, we propose TAF, which provides an iterative execution-based feedback loop that serves as a versatile reasoning scratch-space for task-specific information, from physical material descriptions to layout reasoning. Unlike approaches that attempt to merely extend VLMs into 3D with limited data, the combination of CCF and TAF empowers existing 2D models to reason metrically by grounding 3D understanding visually, without retraining or architectural modification. This design transforms pretrained 2D VLMs into geometry-aware systems capable of interpreting, editing, and generating 3D content.

Building on this foundation, we introduce **3D-Prog**, a 3D reasoning and generation framework that utilizes our task-adaptive and grounded 3D representation. 3D-Prog takes 3D assets such as point clouds, meshes, or Universal Scene Description (USD) scenes along with a natural-language instruction, carries out task-specific analysis, and generates a structured 3D manipulation program. Each line in the program invokes modules for geometric analysis, 3D manipulation, or visual feedback in canonical coordinates. The generated programs consume and produce intermedi-

ate results, forming self-contained reasoning chains that are interpretable and executable. The outputs are written in the USD format, providing a modular, hierarchical, and interpretable representation that is naturally compatible with both 2D VLMs and 3D editing workflows.

Without any retraining or fine-tuning, 3D-Prog performs remarkably well on open-vocabulary 3D understanding, manipulation, and synthesis across object-level and scene-level tasks. It unifies 3D perception, reasoning, and generation in a visually grounded framework, demonstrating that **explicit canonical framing is a prerequisite** for robust geometry-aware multimodal reasoning.

Our key contributions are summarized as follows:

- *Canonical Coordinate Framing (CCF)*: a representation that visually grounds 3D reasoning for 2D VLMs by enforcing a shared Euclidean coordinate system for inputs and outputs, resolving the coordinate-frame ambiguity.
- *Task-Adaptive Feedback (TAF)*: a reasoning scratch-space that enables a CCF-guided VLM to break out of purely geometric reasoning and perform generalist, task-adaptive reasoning over 3D assets.
- *3D-Prog*: a 3D understanding, reasoning, and generation framework building on CCF and TAF. We show that 2D VLMs embedded in 3D-Prog achieve consistent, interpretable, and high-quality results across diverse 3D tasks, without task-specific training or fine-tuning.

2. Related Work

2D VLMs and Emergent Spatial Understanding. Vision-language models (VLMs) [3, 11, 14, 22, 47] have demonstrated remarkable multimodal reasoning capabilities across a wide range of 2D tasks, from image captioning to open-world recognition and VQA. Despite being trained on 2D image-text pairs, these models exhibit emergent spatial and physical understanding: they can ground language in object relations, infer affordances, and even support world models and embodied reasoning frameworks [6, 7, 27, 34, 40]. However, their spatial reasoning remains implicitly encoded within 2D coordinates. As recent evaluations highlight [8, 21, 34, 51], these models lack explicit 3D consistency and cannot directly produce geometry-grounded outputs, limiting their use in 3D reasoning and manipulation.

3D-aware VLMs: Bridging to 3D Understanding. To overcome these limitations, several works have introduced 3D-aware VLMs that explicitly model spatial structure or leverage 3D supervision. Some approaches employ 3D backbones [9, 18–20, 38, 57], e.g., point transformers, while others extend 2D models, e.g., with multi-view consistency [16, 23, 31, 46, 54]. These methods achieve strong grounding in 3D perception and have been applied to both understanding tasks (e.g., 3D captioning, grounding, QA) and generation tasks (e.g., text-to-3D synthesis, scene reconstruction). Despite their promising progress,

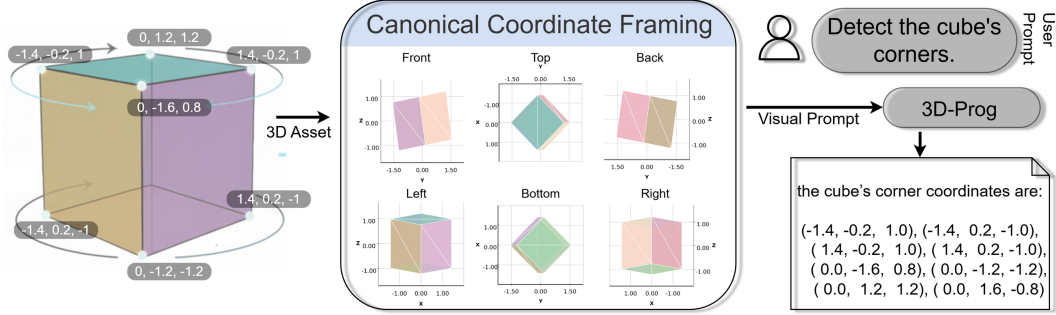


Figure 2. **CCF Overview.** The CCF transforms an input 3D asset into a spatially grounded 2D visual prompt by rendering its six orthographic views, each featuring a fixed origin, visible axes, and metric scale marks.

these 3D-aware VLMs remain constrained by limited multimodal 3D datasets and task-specific training objectives, which hinder their ability to generalize across domains and tasks as broadly as 2D VLMs do [48]. Moreover, existing 3D VLMs typically specialize in either language understanding of 3D assets (e.g., captioning, grounding, VQA) [9, 18, 20, 30, 38, 58] or 3D output generation (e.g., text-to-3D synthesis) [31, 46], rather than offering a unified and flexible framework for both understanding and manipulation.

Visual Programming and Programmatic Reasoning.

Beyond model-centric approaches, visual programming frameworks have emerged as a complementary paradigm for integrating perception, reasoning, and action through modular composition. Neural Module Networks [2] pioneered this direction by decomposing visual question answering into structured, executable programs. Building on this idea, recent works such as VisProg [15], ViperGPT [45], and VADAR [37] demonstrate that complex visual tasks can be represented as modular programs, differing mainly in their approach to program synthesis and API generation. However, this entire line of work remains focused on 2D vision tasks (e.g., detection, VQA, image editing), and thus cannot ingest or operate directly on 3D visual inputs. Efforts towards 3D visual program synthesis have so far been domain-specific, targeting single tasks [32, 36, 53] or specialized 3D asset types [35], which limits their generality. In contrast, 3D-Prog supports diverse 3D assets spanning synthetic and scanned data across varying levels of granularity (object- and scene-level), and enables a wide range of 3D tasks, including generation, editing, and understanding.

3. Method

This section presents *3D-Prog*, our approach for converting 2D VLMs into 3D programmers capable of generation, understanding, manipulation, and reasoning over 3D assets. To enable 2D VLMs to interact with 3D assets, we first pro-

pose a 3D framing protocol, **CCF**, aimed at grounding the 3D inputs to a canonical Euclidean space via images, as explained in Sec. 3.1. To support an open-ended range of 3D tasks, we further propose **TAF**, which allows the model to request and utilize dynamic visual feedback tailored to the instruction, explained in Sec. 3.2. Finally, Sec. 3.3 presents the **3D-Prog** framework, which integrates these components into a unified system capable of performing diverse and complex 3D programming tasks.

3.1. Canonical Coordinate Framing

CCF is a 3D grounding representation strategy that visually anchors both 3D inputs and outputs within a shared global Euclidean coordinate system, as illustrated in Fig. 2. Specifically, we render a single visualization with six orthographic projections of the 3D asset, along with a supplementary perspective view for spatial context. The projections share **a**) visible and labeled X , Y , and Z axes, **b**) metric scale cues via axis markings, and **c**) a fixed reference origin anchored to the asset’s bounding box across all views.

By representing 3D geometry through consistent multi-view visual-linguistic cues, CCF removes gauge ambiguities and establishes a stable and grounded spatial reference for reasoning. CCF shares its principles with various geometric methods such as DUST3R [50], MAST3R [29], and VGGT [49] to achieve multi-view consistency in a shared canonical space, enforcing coherence across viewpoints. Within this canonical frame, the VLM is prompted to express predicted 3D coordinates and spatial relationships directly.

Why Do Other Designs Fail? We observe that designs lacking explicit metric grounding exhibit degenerate behaviors, as summarized in Tab. 1. Without a visible coordinate frame, VLMs struggle to maintain orientation, scale, and metric consistency across views.

By introducing a visible and shared Euclidean reference, **CCF** transforms pretrained 2D VLMs into reliable 3D reasoning engines. It enables **a**) consistent spatial queries (distance, adjacency, containment), **b**) geometry-aware visual

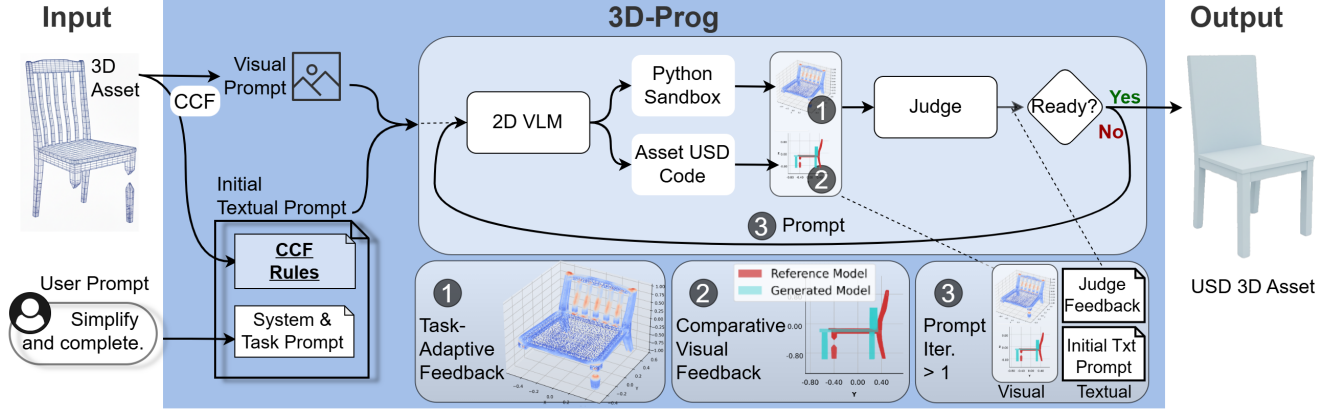


Figure 3. **3D-Prog Overview.** Before being passed to 3D-Prog, the user input is pre-processed by the CCF. Within 3D-Prog, a 2D VLM generates Python code for both (1) Task-Adaptive Analysis and (2) asset generation. A Judge module evaluates this feedback and either confirms the task is complete or provides (3) Iterative Prompt Feedback, comprising both visual and textual cues, which is fed back to the 2D VLM for iterative refinement.

Table 1. **Failure analysis of ungrounded designs.** Explicit metric grounding is essential for stable and interpretable 3D reasoning.

Missing Design Element	Observed Failure Mode
No visible axes	Orientation ambiguity
No axis markings or values	Inconsistent scale
Perspective-only projections	Metric distortion
Parametric-only 3D outputs	Floating reference frame

grounding across dialogue turns, c) controllable and interpretable 3D scene editing, and d) multi-view alignment without pose metadata.

We conclude that 3D grounding through explicit canonical framing is a prerequisite for robust geometry-aware multimodal interaction.

3.2. Task-Adaptive Feedback

A key capability of a generalist 3D programmer is the ability to request and obtain task-relevant feedback. While CCF enables seamless interaction between 3D assets and VLMs, the generated asset may still fall short of the task requirements.

To bridge this gap, we propose **TAF**, a mechanism that establishes a task-dependent feedback loop between the VLM and its generated content. Concretely, we link the VLM to a Python execution environment, enabling it to produce TAF as visual feedback. In each iteration, the VLM generates USD manipulation code and task-specific analysis code that programmatically inspects the current 3D asset, producing diagnostic visualizations (e.g., height distributions, density heatmaps, segmentation overlays). These visual outputs are rendered and fed back to the VLM alongside the updated CCF views of the generated asset, form-

ing an iterative loop in which the VLM identifies discrepancies, refines its hypotheses, and corrects its output based on concrete evidence from the asset itself. The VLM autonomously determines what intermediate analysis is needed for a given task, allowing TAF to range from plotting material properties and assessing physical plausibility to running non-trivial operations such as DBSCAN clustering to isolate object parts for segmentation (see Fig. 5).

3.3. 3D-Prog Framework

Building upon CCF and TAF, we introduce 3D-Prog, a generalist 3D programming framework that leverages the broad semantic and reasoning capabilities of the 2D VLM to operate effectively on 3D assets. 3D-Prog transforms VLMs into 3D programmers capable of carrying out a wide range of tasks without requiring task-specific supervision or adaptation. Our framework is designed around three core principles: (1) CCF expresses 3D assets in a canonical frame, providing grounding cues that enable VLMs trained on 2D data to infer spatial relationships; (2) TAF equips the system with an iterative feedback mechanism that closes the loop between generation and evaluation; and (3) the USD format, with its hierarchical and compositional nature, aligns scene representation with VLM reasoning patterns, enabling complex 3D transformations to be decomposed into interpretable programmatic steps.

Following these principles, the architecture comprises three primary modules:

2D VLM for perception: CCF-rendered views of the 3D assets serve as the perceptual input to the VLM. Conditioned on these renders and the task description, the VLM produces a) semantic and geometric interpretations of the scene and b) executable code that governs the manipulation of the 3D asset. The generated code is executed inside a

Python environment, which provides the essential capabilities of TAF and USD manipulation, enabling the VLM to construct, edit, or transform the underlying USD scene via OpenUSD in Python.

Judge: To provide critical feedback, a separate Judge VLM evaluates the proposed solution. The generated USD asset is rendered and overlaid with the reference model’s CCF representation (Sec. 3.1) to provide iterative visual feedback to the VLM. The judge evaluates the solution based on the user task prompt, comparative visual feedback, and TAF-produced visual analysis. The evaluation yields a structured response comprising a binary completion status, a confidence score, and a diagnostic report (a feedback comment alongside lists of detected errors and prescriptive suggestions). The judge response is appended to the Python output to guide the VLM in subsequent iterations.

Iterative Refinement through TAF: The judge feedback and TAF-produced diagnostic visualizations are fed back to the VLM, which generates an updated USD asset in the next iteration. This TAF-driven loop allows the VLM to correct trivial mistakes (such as faulty USD code, incorrect axes, or wrong rotations), improve geometric fidelity based on visual evidence from its own analysis, and progressively refine its output from coarse to fine-grained solutions. The process continues until the task is successfully completed or a predefined maximum number of iterations is reached.

3.4. Generalization of 3D-Prog

3D-Prog handles a wide range of 3D vision tasks within a unified framework, spanning three functional domains: **generation** (object completion, Scan2Sim), **understanding** (detection, segmentation, editing, QA, captioning), and **reasoning** (compositional scene building, articulation prediction). A detailed task breakdown is provided in the Supplementary Material, Tab. S1.

By expressing all outputs as USD code, diverse 3D objectives become instances of the same composable representation that can be inspected, edited, and executed across standard USD-based toolchains. This unified abstraction allows a single system to analyze, modify, and construct 3D scenes from natural-language or visual instructions, positioning USD as a lingua franca for 3D understanding and generation. The range of tasks spans both synthetic and real-world 3D assets at object and scene scales, demonstrating the generalization of 3D-Prog across asset types.

Within 3D generation, the system performs tasks that synthesize or transform raw geometry into simulation-ready assets, such as object completion and “Scan2Sim” reconstruction. To facilitate 3D understanding, 3D-Prog interprets and structures data across geometrical, spatial, visual, contextual, and semantic-spatial dimensions, enabling tasks such as object detection or object editing. Finally, reasoning tasks are a core capability, e.g., predicting articulation

Table 2. Comparison of 3D-Prog with closed (Gemini 2.5 Pro, GPT-5.2, Claude Sonnet 4.5) and open (Kimi K2.5) 2D VLMs on generation, understanding, and reasoning tasks in 3D. 3D-Prog (C) and (K) use Claude and Kimi as backbones. ✗ denotes failure to produce valid USD outputs.

Task	Closed Models				Open Models	
	Gemini	GPT	Claude	3D-Prog (C)	Kimi	3D-Prog (K)
Generation						
Object Compl. (Chamfer ↓)	1.17	0.51	0.42	0.19	0.45	0.16
Scan2Sim (Chamfer ↓)	✗	✗	✗	0.08	✗	0.09
Understanding						
Object Det. (mAP ↑)	0.00	0.16	0.04	0.38	0.02	0.04
Instance Seg. (mAP ↑)	0.18	0.28	0.21	0.42	0.22	0.43
Semantic Seg. (mIoU ↑)	0.48	0.47	0.48	0.53	0.47	0.52
Object Editing (User ↑)	1.40	1.40	4.00	4.80	3.59	3.59
Reasoning						
Composition (User ↑)	2.70	2.35	2.00	4.60	3.00	3.70
Articulation (Axis Err ↓)	86.58	90.00	86.94	3.50	89.61	0.64

and composing coherent scenes from discrete objects.

4. Experiments

We evaluate 3D-Prog through task benchmarks, ablation studies, experiments across diverse VLM families, and spatial robustness analysis. We demonstrate that 2D VLMs, when grounded via CCF and paired with TAF, can perform diverse 3D programming tasks effectively.

Datasets. We evaluate 3D-Prog on diverse 3D programming tasks spanning generation, understanding, and reasoning (Tab. S1), covering both object- and scene-level settings with synthetic and real-world data. For Scan2Sim and articulation prediction, we use real-world scans and annotations from the Articulate3D dataset [17] at the object and scene levels, respectively. Semantic and instance part segmentation, object editing, and object completion use synthetic assets from Objaverse-XL [13] and Pixar’s OpenUSD asset library [44], which provide ground-truth part segmentations and semantic labels. For object editing, we curate assets to maximize geometric and visual complexity; for object completion, inputs are generated by removing pre-segmented object parts. For object detection, we combine synthetic data [44] with high-quality laser scans from ScanNet++ [52], which includes human-annotated segmentations. Finally, compositional reasoning is evaluated using five natural-language queries describing complex multi-object indoor environments. The full system and judge prompts are provided in the supplementary material.

Metrics. We evaluate 3D-Prog using task-specific metrics as well as human judgment to capture both geometric fidelity and semantic plausibility. For object-level perception, we report mean Average Precision (mAP) for detection and instance segmentation and mean Intersection over Union (mIoU) for semantic segmentation. Geometric re-

construction quality for object completion and Scan2Sim is quantified using Chamfer distance, while articulation accuracy is measured via the median motion axis error. For object editing and compositional reasoning, where automated metrics often fail to capture nuance, we conduct a user study with four participants. Each instance is evaluated against five task-specific success criteria and scored on a 0–5 scale. We report the mean score aggregated across all samples and evaluators. Detailed success criteria and user study protocols are provided in the supplementary material (Sec. C).

Setup. We evaluate task performance across a diverse selection of 2D VLMs, including the proprietary systems Claude Sonnet 4.5 [3], Gemini 2.5 Pro [11], and GPT-5.2 [41], as well as the open-source Kimi K2.5 [28] model. Additionally, we instantiate open-source and closed-source variants of 3D-Prog using the top-performing open-source and proprietary VLMs as their respective foundational cores. The iterative refinement process is capped at 20 iterations, with an early stopping condition triggered upon approval by the judge. If the iteration limit is reached, the terminal output is recorded as the final result. Performance for each task is quantified as the mean score across five distinct assets.

Baselines and Ablations. In the main comparison (Tab. 2), we evaluate four standalone 2D VLMs as baselines: Gemini 2.5 Pro, GPT-5.2, Claude Sonnet 4.5, and Kimi K2.5. We instantiate 3D-Prog with Claude and Kimi as backbones, as they are the overall top-performing closed-source and open-source baselines, respectively. The primary distinction between a baseline and its 3D-Prog variant is the omission of CCF and TAF; baselines are otherwise granted identical task prompts. Claude Sonnet 4.5 is additionally used as the dedicated baseline for ablation studies, robustness testing, and 3D VLM comparisons, as it demonstrated the highest aggregate performance among all evaluated 2D VLMs.

4.1. Results and Analysis

We evaluate 3D-Prog across the tasks introduced in Sec. 3.4 against diverse VLM baselines. Shown in Tab. 2, 3D-Prog consistently outperforms or matches both proprietary and open-source baselines across all benchmarks, demonstrating how 3D-Prog equips VLMs with 3D generative, understanding, and reasoning capabilities.

For generative tasks, 3D-Prog is able to simplify and complete both dense complex object scans and synthetic assets (e.g., 0.19 Chamfer distance for synthetic data completion and 0.08 for Scan2Sim), indicating high geometric fidelity. The Scan2Sim task shows a critical point of model capabilities, as all baseline models proved incapable of generating valid USD files from real-world data. These failures stem from either (a) non-compliance with simplification instructions, leading to context window overflow during the

Table 3. Ablation results on diverse tasks spanning generation and understanding.

	3D-Prog	Claude	w/o CCF	w/o TAF
Instance Segmentation (mAP $\uparrow$)	0.42	0.21	0.30	0.29
Object Detection (mAP $\uparrow$)	0.38	0.04	0.01	0.11
Object Completion (Chamfer $\downarrow$)	0.19	0.42	0.33	0.58

ingestion of dense meshes, or (b) catastrophic geometric loss caused by the models’ inability to reconcile real-world scan noise and topological inconsistencies.

3D-Prog further demonstrates strong performance on 3D understanding tasks. Object detection and instance segmentation performance nearly double, confirming significantly improved 3D spatial reasoning at both the scene and object levels. Furthermore, user study results for object editing favor 3D-Prog, indicating capabilities that extend beyond pure geometric understanding.

3D-Prog exhibits robust spatial reasoning, evidenced by its articulation predictions. It correctly identifies the motion type in every case and predicts the motion axis with a median error of 3.5° and 0.6° with the Claude and Kimi backbones, whereas standalone 2D VLMs err by nearly 90° . In the compositional reasoning task, evaluators show a strong preference for the scenes generated by 3D-Prog, highlighting their superior geometric and visual quality. Assets generated by 3D-Prog exhibit clean geometry with no part or object-level intersections, representative shapes for each object class, and realistic scale, materials, and color, leading to visually coherent and physically plausible compositions. In contrast, baseline outputs often contain non-distinctive geometry, unrealistic element proportions, detached components, and frequent mesh intersections between separate parts, resulting in visually unstable or semantically inconsistent scenes. Qualitative comparisons are shown in Fig. 4, with detailed analysis of evaluator feedback from both user studies provided in the Suppl. material (Sec. C).

4.2. Ablation Study

We conduct ablation studies to examine the role of the CCF and TAF across various asset categories and both understanding and generative tasks, as shown in Tab. 3. Specifically, we assess 3D-Prog across three distinct benchmarks: (a) instance segmentation, to evaluate fine-grained geometric and object-level comprehension; (b) object detection, to probe scene-level understanding; and (c) object completion, to demonstrate the model’s integrated capacity for geometric analysis and generative synthesis. The results confirm that CCF and TAF are complementary. CCF provides the geometric grounding essential for metric reasoning—without it, completion Chamfer distance worsens from 0.19 to 0.33 and detection mAP collapses from 0.38 to 0.01. TAF enables iterative error correction; its removal eliminates the

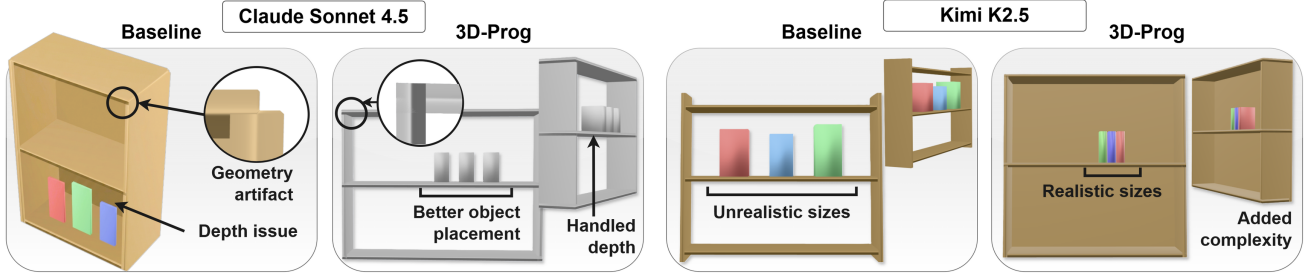


Figure 4. Qualitative results of 3D-Prog with Claude and Kimi backbones vs Baselines on Compositional Reasoning. Prompt: “Generate a simple bookshelf with 3 books.”

Table 4. **Robustness to geometric perturbations.** Chamfer distance ($\downarrow$) on an object completion asset when the input and its ground truth are translated along Y or rotated about Z ; 3D-Prog (Claude backbone) vs. the Claude Sonnet 4.5 baseline. The average Chamfer distance is calculated over the six perturbed settings.

Method	Original	Translated (Y)			Rotated (Z)			Avg.
		+10cm	+20cm	+30cm	+15°	+30°	+45°	
3D-Prog	0.02	0.02	0.02	0.02	0.02	0.07	0.18	0.05
Claude	0.02	0.38	0.44	0.11	0.33	0.18	0.17	0.27

refinement loop, causing a $3\times$ degradation (0.19 to 0.58). This disentangles the contribution: CCF grounds spatial understanding, while TAF provides the iterative scaffold for complex tasks. Specifically, omitting CCF restricts the visual prompt to a single orthographic view without coordinate framing, introducing inaccurate initial coordinate estimates that the VLM cannot rectify. Similarly, omitting TAF removes the task-adaptive analysis that drives the iterative refinement loop, reducing the system to a single generation pass. Without iterative analysis, initial noisy outputs remain uncorrected, resulting in a sharp performance decline.

Robustness to geometric perturbations. To test the stability of CCF grounding, we translate and rotate the input of the object completion task (transforming the ground truth accordingly) and compare against the Claude baseline in Tab. 4. Both methods are equally accurate on the unperturbed input, but 3D-Prog holds a constant 0.02 Chamfer distance under every translation and the 15° rotation, and 0.07 at 30°, whereas the baseline degrades by up to $22\times$ and fluctuates unpredictably between perturbation magnitudes. Only at 45° do both methods degrade similarly.

4.3. 3D Programming Across VLM Families

To evaluate how broadly 3D-Prog can transform 2D VLMs into 3D programmers, we benchmark it on various closed-source and open-source 2D VLMs. As shown in Tab. 5, 3D-Prog yields consistent improvements across diverse VLM families, both closed-source and open-source.

Open-source vs closed-source VLMs for 3D program-

Table 5. **3D-Prog across various VLMs** on object completion, with state-of-the-art open- and closed-source 2D VLMs as backbones.

Metric	Baseline	3D-Prog (Ours)			
	Sonnet 4.5	 GPT-5.1 OpenAI	 Gemini 3 Pro Gemini	 Sonnet 4.5 Claude	 Kimi K2.5 KIMI
Chamfer ↓	0.42	0.23	0.25	0.19	0.16

Table 6. **Comparisons to 3D VLMs.** 3D-Prog outperforms GaussianVLM [18] on both SQA3D and Nr3D.

	Claude	GaussianVLM	3D-Prog
SQA3D (Accuracy $\uparrow$)	30%	50%	60%
Nr3D (METEOR $\uparrow$)	0.16	0.22	0.23

ming. Our benchmark of open-source VLMs includes Kimi K2.5, Llama 4 [39], and Qwen3-VL-30B-A3B-Instruct [4]. Of these models, only Kimi K2.5 demonstrates the required Python code generation capabilities and adaptability to the USD format. The failure of other models to produce valid outputs or process 3D data highlights a gap in open-source models for geometry-aware code generation. Although open-source VLMs like Qwen and Llama have sufficient visual understanding, they lack adequate coding/programming expertise for these tasks. However, the promising results from Kimi K2.5 suggest that high-performance 3D programming is becoming increasingly attainable within open-source frameworks.

4.4. Comparisons to 3D VLMs

We evaluate the performance of 3D-Prog against a state-of-the-art 3D VLM [18] across two primary dimensions: scene-level and object-level 3D understanding. For scene-level evaluation, we assess the model’s ability to perform situated reasoning using the SQA3D benchmark [34], which requires interpreting complex environments from a specified egocentric perspective (e.g., “I am sitting on the bed facing the chair. What is to my direct left?”). Object-

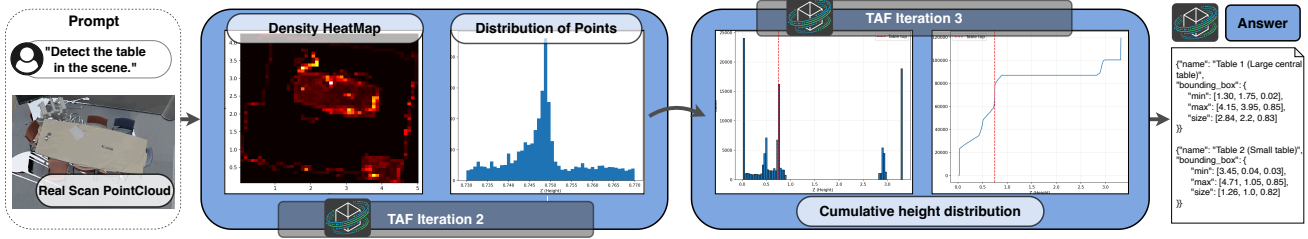


Figure 5. **TAF across iterations** for table detection in a scanned scene. Early iterations produce two cues: a density heatmap of points at typical table height (candidate footprints) and a height distribution (expected vertical extent). The next iteration restricts the analysis to the extracted footprint and uses the height distribution and its cumulative counterpart for noise rejection, yielding the final 3D bounding box.

level capabilities are evaluated on object captioning, where the model is provided with object bounding boxes and tasked with generating captions for the corresponding objects using the Nr3D benchmark [1]. Performance is measured using accuracy for SQA3D and the METEOR [5] captioning metric for Nr3D. We compare 3D-Prog with the Claude 2D VLM baseline and GaussianVLM, a generalist 3D VLM [18]. Results in Tab. 6 show that our training-free approach outperforms the state-of-the-art 3D VLM specifically trained on these datasets and tasks.

4.5. Discussion of Experiments

The analysis of 3D-Prog reveals that CCF is essential for tasks fundamentally rooted in geometric reasoning, e.g., object completion or compositional generation. In these settings, resolving gauge ambiguity and providing spatial grounding enable 3D-Prog to converge to a solution consistently over multiple iterations. As expected, for tasks that primarily require appearance changes without geometric adaptations (e.g., simple object editing), the baseline model can achieve a reasonable success score by mapping the task to the 2D domain. Nevertheless, even in these scenarios, our approach outperforms the baseline, showcasing that additional feedback is consistently beneficial.

In a detailed analysis, we observe that visual feedback produced by the TAF module shows a strong degree of specialization tailored to the specific user query and task category. TAF demonstrates the ability to construct specialized visual feedback, generate complex NumPy code, and employ classical ML algorithms such as clustering for segmentation tasks. Furthermore, TAF exhibits an inherently evolutionary nature: across successive iterations, its outputs become increasingly fine-grained by leveraging insights from preceding passes. This evolution from broad analysis to focused refinement is exemplified in the table detection task (Fig. 5). The first iteration generates general cues based on universal rules, while in subsequent iterations, the model shifts to focused asset-specific region analysis. Additional qualitative examples of TAF and a comprehensive failure analysis for both 3D-Prog and the baseline are provided in

the Supplementary Material (Sec. D, E).

Across evaluation tasks, 3D-Prog converges in 7–20 iterations (mean: 11), with each iteration taking approximately 22s on average. Total runtime ranges from around 2 minutes for articulation and QA tasks to 8 minutes for segmentation. VLM inference dominates compute, followed by code execution and rendering. A detailed per-task breakdown is provided in the Supplementary Material.

5. Conclusion

In this work, we presented 3D-Prog, a generalizable approach that enables 2D VLMs to perform 3D tasks, including understanding, reasoning, editing, composition, and generation. 3D-Prog achieves this through CCF, which enables grounding and 3D perception in 2D VLMs, and TAF, which encompasses a flexible scratch-space for task-specific feedback. Experiments further showcased that 3D-Prog is a powerful generalist framework that brings the capabilities of current 2D VLMs to free-form 3D programming, unlocking the potential of VLMs for 3D tasks. We demonstrated the versatility and generalizability of 3D-Prog across a diverse set of tasks on both real and synthetic assets, where it consistently and reliably produces high-quality 3D outputs and achieves precise 3D understanding.

Limitations. Although powerful, 3D-Prog is not without limitations. First, by design, the performance of 3D-Prog depends on the capabilities of the underlying VLMs, which still struggle with very complex reasoning tasks or fine-grained details. Second, 3D-Prog relies on a judge module that evaluates convergence rather than correctness. While 100% motion-type accuracy on articulation suggests the iterative loop is self-correcting, the judge may occasionally accept suboptimal results. We mitigate this through conservative iteration limits and visual comparison feedback. Lastly, 3D-Prog’s ability to process 3D assets relies on the CCF, which is limited in scene size and detail by the VLM context. However, with the fast progress in open and closed VLMs, our method has a clear path towards scaling beyond its current performance.

Acknowledgments

This research was partially funded by the Ministry of Education and Science of Bulgaria (support for INSAIT, part of the Bulgarian National Roadmap for Research Infrastructure).

References

- [1] P. Achlioptas et al. ReferIt3D: Neural listeners for fine-grained 3d object identification in real-world scenes. *ECCV*, 2020. 8, 13
- [2] Jacob Andreas, Marcus Rohrbach, Trevor Darrell, and Dan Klein. Neural module networks. *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 39–48, 2015. 3
- [3] Anthropic. Introducing claude sonnet 4.5. <https://www.anthropic.com/news/claude-sonnet-4-5>, 2025. Accessed: 2025-11-13. 2, 6
- [4] Shuai Bai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, Xionghui Chen, Zesen Cheng, Lianghao Deng, Wei Ding, Chang Gao, Chunjiang Ge, Wenbin Ge, Zhifang Guo, Qidong Huang, Jie Huang, Fei Huang, Binyuan Hui, Shutong Jiang, Zhao-hai Li, Mingsheng Li, Mei Li, Kaixin Li, Zicheng Lin, Junyang Lin, Xuejing Liu, Jiawei Liu, Chenglong Liu, Yang Liu, Dayiheng Liu, Shixuan Liu, Dunjie Lu, Ruilin Luo, Chenxu Lv, Rui Men, Lingchen Meng, Xuancheng Ren, Xingzhang Ren, Sibao Song, Yuchong Sun, Jun Tang, Jianhong Tu, Jianqiang Wan, Peng Wang, Pengfei Wang, Qiuyue Wang, Yuxuan Wang, Tianbao Xie, Yiheng Xu, Haiyang Xu, Jin Xu, Zhibo Yang, Mingkun Yang, Jianxin Yang, An Yang, Bowen Yu, Fei Zhang, Hang Zhang, Xi Zhang, Bo Zheng, Humen Zhong, Jingren Zhou, Fan Zhou, Jing Zhou, Yuanzhi Zhu, and Ke Zhu. Qwen3-vl technical report, 2025. 7
- [5] Satanjeev Banerjee and Alon Lavie. Meteor: An automatic metric for mt evaluation with improved correlation with human judgments. *ACL workshop on intrinsic and extrinsic evaluation measures for machine translation and/or summarization*, 2005. 8
- [6] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024. 2
- [7] Jake Bruce, Michael D Dennis, Ashley Edwards, Jack Parker-Holder, Yuge Shi, Edward Hughes, Matthew Lai, Aditi Mavalankar, Richie Steigerwald, Chris Apps, et al. Genie: Generative interactive environments. In *Forty-first International Conference on Machine Learning*, 2024. 2
- [8] Boyuan Chen, Zhuo Xu, Sean Kirmani, Brain Ichter, Dorsa Sadigh, Leonidas Guibas, and Fei Xia. Spatialvlm: Endowing vision-language models with spatial reasoning capabilities. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 14455–14465, 2024. 2
- [9] Sijin Chen, Xin Chen, Chi Zhang, Mingsheng Li, Gang Yu, Hao Fei, Hongyuan Zhu, Jiayuan Fan, and Tao Chen. Ll3da: Visual interactive instruction tuning for omni-3d understanding reasoning and planning. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 26428–26438, 2024. 2, 3
- [10] Xingyu Chen, Fu-Jen Chu, Pierre Gleize, Kevin J Liang, Alexander Sax, Hao Tang, Weiyao Wang, Michelle Guo, Thibaut Hardin, Xiang Li, et al. Sam 3d: 3dfy anything in images. *arXiv preprint arXiv:2511.16624*, 2025. 15
- [11] Gheorghe Comanici, Eric Bieber, Mike Schaekermann, Ice Pasupat, Noveen Sachdeva, Inderjit Dhillon, Marcel Blisstein, Ori Ram, Dan Zhang, Evan Rosen, et al. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *arXiv preprint arXiv:2507.06261*, 2025. 2, 6
- [12] Angela Dai, Angel X. Chang, Manolis Savva, Maciej Halber, Thomas A. Funkhouser, and Matthias Nießner. Scannet: Richly-annotated 3d reconstructions of indoor scenes. *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2432–2443, 2017. 12, 13
- [13] Matt Deitke, Ruoshi Liu, Matthew Wallingford, Huong Ngo, Oscar Michel, Aditya Kusupati, Alan Fan, Christian Laforte, Vikram Voleti, Samir Yitzhak Gadre, et al. Objaverse-xl: A universe of 10m+ 3d objects. In *Advances in Neural Information Processing Systems*, 2023. 5, 13
- [14] Matt Deitke, Christopher Clark, Sangho Lee, Rohun Tripathi, Yue Yang, Jae Sung Park, Mohammadreza Salehi, Niklas Muennighoff, Kyle Lo, Luca Soldaini, et al. Molmo and pixmo: Open weights and open data for state-of-the-art vision-language models. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 91–104, 2025. 2
- [15] Tanmay Gupta and Aniruddha Kembhavi. Visual programming: Compositional visual reasoning without training. *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 14953–14962, 2022. 3
- [16] Huy Ha and Shuran Song. Semantic abstraction: Open-world 3d scene understanding from 2d vision-language models. In *Conference on Robot Learning*, 2022. 2
- [17] Anna-Maria Halacheva, Yang Miao, Jan-Nico Zaeck, Xi Wang, Luc Van Gool, and Danda Pani Paudel. Articulate3d: Holistic understanding of 3d scenes as universal scene description. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2025. 2, 5, 13
- [18] Anna-Maria Halacheva, Jan-Nico Zaeck, Xi Wang, Danda Pani Paudel, and Luc Van Gool. Gaussianvlm: Scene-centric 3d vision-language models using language-aligned gaussian splats for embodied reasoning and beyond. *IEEE Robotics Autom. Lett.*, 10(12):12588–12595, 2025. 2, 3, 7, 8
- [19] Yining Hong, Haoyu Zhen, Peihao Chen, Shuhong Zheng, et al. 3d-llm: Injecting the 3d world into large language models. *NeurIPS*, 2023.
- [20] Haifeng Huang, Yilun Chen, Zehan Wang, Rongjie Huang, Runsen Xu, et al. Chat-scene: Bridging 3d scene and large language models with object identifiers. *NeurIPS*, 2024. 2, 3
- [21] Jiangyong Huang, Silong Yong, Xiaojian Ma, Xiongkun Linghu, Puhao Li, et al. An embodied generalist agent in 3d world. In *ICML*, 2024. 2

- [22] Aaron Hurst, Adam Lerer, Adam P Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, et al. Gpt-4o system card. *arXiv preprint arXiv:2410.21276*, 2024. 2
- [23] Krishna Murthy Jatavallabhula, Alihusein Kuwajerwala, Qiao Gu, Mohd Omama, Tao Chen, Shuang Li, Ganesh Iyer, Soroush Saryazdi, Nikhil Keetha, Ayush Tewari, Joshua B. Tenenbaum, Celso Miguel de Melo, Madhava Krishna, Liam Paull, Florian Shkurti, and Antonio Torralba. Conceptfusion: Open-set multimodal 3d mapping. *Robotics: Science and Systems (RSS)*, 2023. 2
- [24] Baoxiong Jia, Yixin Chen, Huangyue Yu, Yan Wang, Xuesong Niu, Tengyu Liu, Qing Li, and Siyuan Huang. Sceneverse: Scaling 3d vision-language learning for grounded scene understanding. In *European Conference on Computer Vision (ECCV)*, 2024. 2
- [25] Mukul Khanna, Yongsen Mao, Hanxiao Jiang, Sanjay Haresh, Brennan Shacklett, Dhruv Batra, Alexander Clegg, Eric Undersander, Angel X Chang, and Manolis Savva. Habitat synthetic scenes dataset (hssd-200): An analysis of 3d scene scale and realism tradeoffs for objectgoal navigation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 16384–16393, 2024. 2
- [26] Hyunjin Kim and Minhyuk Sung. Partstad: 2d-to-3d part segmentation task adaptation. In *European Conference on Computer Vision*, 2024. 2
- [27] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan P Foster, Pannag R Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. OpenVLA: An open-source vision-language-action model. In *8th Annual Conference on Robot Learning*, 2024. 2
- [28] Kimi. Kimi k2.5: Visual agentic intelligence. <https://www.kimi.com/blog/kimi-k2-5.html>, 2026. 6
- [29] Vincent Leroy, Yohann Cabon, and Jerome Revaud. Grounding image matching in 3d with mast3r, 2024. 3
- [30] Mingsheng Li, Xin Chen, Chi Zhang, Sijin Chen, Hongyuan Zhu, et al. M3dbench: Towards omni 3d assistant with interleaved multi-modal instructions. *ECCV*, 2025. 3
- [31] Yangguang Li, Zi-Xin Zou, Zexiang Liu, Dehu Wang, Yuan Liang, Zhipeng Yu, Xingchao Liu, Yuan-Chen Guo, Ding Liang, Wanli Ouyang, et al. Triposg: High-fidelity 3d shape synthesis using large-scale rectified flow models. *arXiv preprint arXiv:2502.06608*, 2025. 2, 3
- [32] Gabrielle Littlefair, Niladri Shekhar Dutt, and Niloy J Mitra. Flairgpt: Repurposing llms for interior designs. *arXiv preprint arXiv:2501.04648*, 2025. 3
- [33] Anran Liu, Cheng Lin, Yuan Liu, Xiaoxiao Long, Zhiyang Dou, Hao-Xiang Guo, Ping Luo, and Wenping Wang. Part123: Part-aware 3d reconstruction from a single-view image. *ACM SIGGRAPH 2024 Conference Papers*, 2024. 2
- [34] Xiaojian Ma, Silong Yong, Zilong Zheng, Qing Li, Yitao Liang, et al. Sqa3d: Situated question answering in 3d scenes. In *ICLR*, 2023. 2, 7, 13
- [35] Dimitrios Mallis, Ahmet Serda Karadeniz, Sebastian Cavada, Danila Rukhovich, Niki Foteinopoulou, Kseniya Cherenkova, Anis Kacem, and Djamila Aouada. Cad-assistant: tool-augmented vllms as generic cad task solvers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 7284–7294, 2025. 3
- [36] Yongsen Mao, Junhao Zhong, Chuan Fang, Jia Zheng, Rui Tang, Hao Zhu, Ping Tan, and Zihan Zhou. Spatiallm: Training large language models for structured indoor modeling. In *Advances in Neural Information Processing Systems*, 2025. 3
- [37] Damiano Marsili, Rohun Agrawal, Yisong Yue, and Georgia Gkioxari. Visual agentic ai for spatial reasoning with a dynamic api. In *Proceedings of the Computer Vision and Pattern Recognition Conference (CVPR)*, pages 19446–19455, 2025. 3
- [38] Guofeng Mei, Wei Lin, Luigi Riz, Yujiao Wu, Fabio Poiesi, and Yiming Wang. Perla: Perceptive 3d language assistant. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 14369–14379, 2025. 2, 3
- [39] Meta. The llama 4 herd: The beginning of a new era of natively multimodal ai innovation. <https://ai.meta.com/blog/llama-4-multimodal-intelligence/>, 2025. Accessed: 2026-03-01. 7
- [40] Octo Model Team, Dibya Ghosh, Homer Walke, Karl Pertsch, Kevin Black, Oier Mees, Sudeep Dasari, Joey Hejna, Charles Xu, Jianlan Luo, Tobias Kreiman, You Liang Tan, Lawrence Yunliang Chen, Pannag Sanketi, Quan Vuong, Ted Xiao, Dorsa Sadigh, Chelsea Finn, and Sergey Levine. Octo: An open-source generalist robot policy. In *Proceedings of Robotics: Science and Systems*, Delft, Netherlands, 2024. 2
- [41] OpenAI. Introducing gpt-5.2. <https://openai.com/index/introducing-gpt-5-2/>, 2025. Accessed: 2026-03-02. 6
- [42] Songyou Peng, Kyle Genova, Chiyu "Max" Jiang, Andrea Tagliasacchi, Marc Pollefeys, and Thomas Funkhouser. Openscene: 3d scene understanding with open vocabularies. In *CVPR*, 2023. 2
- [43] Yawen Shao, Wei Zhai, Yuhang Yang, Hongchen Luo, Yang Cao, and Zheng-Jun Zha. Great: Geometry-intention collaborative inference for open-vocabulary 3d object affordance grounding. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 17326–17336, 2025. 2
- [44] Pixar Animation Studios. Kitchen set – universal scene description (usd). https://openusd.org/release/dl_kitchen_set.html, 2021. Accessed: 2025-11-13. 5, 13
- [45] D’idac Sur’is, Sachit Menon, and Carl Vondrick. Vipergpt: Visual inference via python execution for reasoning. 2023 *IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 11854–11864, 2023. 3
- [46] Tencent Hunyuan3D Team. Hunyuan3d 2.0: Scaling diffusion models for high resolution textured 3d assets generation, 2025. 2, 3
- [47] Michael Tschannen, Alexey Gritsenko, Xiao Wang, Muhammad Ferjad Naeem, Ibrahim Alabdulmohsin, et al. Siglip

- 2: Multilingual vision-language encoders with improved semantic understanding, localization, and dense features. *arXiv:2502.14786*, 2025. [2](#)
- [48] Austin Wang, ZeMing Gong, and Angel X Chang. Vigil3d: A linguistically diverse dataset for 3d visual grounding. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 30453–30475, 2025. [3](#)
- [49] Jianyuan Wang, Minghao Chen, Nikita Karaev, Andrea Vedaldi, Christian Rupprecht, and David Novotny. Vggt: Visual geometry grounded transformer. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 5294–5306, 2025. [3](#)
- [50] Shuzhe Wang, Vincent Leroy, Yohann Cabon, Boris Chidlovskii, and Jerome Revaud. Dust3r: Geometric 3d vision made easy. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 20697–20709, 2024. [3](#)
- [51] Jihan Yang, Shusheng Yang, Anjali W Gupta, Rilyn Han, Li Fei-Fei, and Saining Xie. Thinking in space: How multimodal large language models see, remember, and recall spaces. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 10632–10643, 2025. [2](#)
- [52] Chandan Yeshwanth, Yueh-Cheng Liu, Matthias Nießner, and Angela Dai. Scannet++: A high-fidelity dataset of 3d indoor scenes. In *ICCV*, 2023. [5](#), [12](#), [13](#)
- [53] Haocheng Yuan, Jing Xu, Hao Pan, Adrien Bousseau, Niloy J Mitra, and Changjian Li. Cadtalk: An algorithm and benchmark for semantic commenting of cad programs. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 3753–3762, 2024. [3](#)
- [54] Renrui Zhang, Ziyu Guo, Wei Zhang, Kunchang Li, Xupeng Miao, Bin Cui, Yu Qiao, Peng Gao, and Hongsheng Li. Pointclip: Point cloud understanding by clip. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 8552–8562, 2022. [2](#)
- [55] Mingrui Zhao, Yizhi Wang, Fenggen Yu, Changqing Zou, and Ali Mahdavi-Amiri. Sweepnet: Unsupervised learning shape abstraction via neural sweepers. In *European Conference on Computer Vision*, 2024. [2](#)
- [56] Hongyan Zhi, Peihao Chen, Junyan Li, Shuailei Ma, Xinyu Sun, Tianhang Xiang, Yinjie Lei, Minghui Tan, and Chuang Gan. Lscenellm: Enhancing large 3d scene understanding using adaptive visual preferences. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 3761–3771, 2025. [2](#)
- [57] Ziyu Zhu, Xiaojian Ma, Yixin Chen, Zhidong Deng, et al. 3d-vista: Pre-trained transformer for 3d vision and text alignment. *CVPR*, 2023. [2](#)
- [58] Ziyu Zhu, Zhuofan Zhang, Xiaojian Ma, Xuesong Niu, Yixin Chen, Baoxiong Jia, Zhidong Deng, Siyuan Huang, and Qing Li. Unifying 3d vision-language understanding via promptable queries. *ECCV*, 2024. [3](#)

Supplementary Material

We provide an extended analysis and implementation details to support the main manuscript. The supplementary material is organized as follows: Sec. A provides a detailed task taxonomy and dataset licenses. Sec. B reports per-task runtime statistics. Sec. C presents the methodology of our user study. We then provide a detailed TAF analysis (Sec. D), followed by a qualitative error analysis (Sec. E) to investigate current limitations. We further expand the scope of our evaluation with qualitative results in Sec. F. We conclude by detailing the specific design choices (Sec. G), including the core algorithm and full prompt specifications.

A. Task Overview

We demonstrate 3D-Prog across nine tasks spanning generation, understanding, and reasoning. Tab. S1 provides a detailed breakdown of each task, its description, and input asset types.

Dataset Licenses and Terms of Use. We report the licenses and terms of use for the datasets used in our evaluation in Tab. S2 and adhere strictly to each. All datasets are publicly available. We draw attention to ScanNet [12]¹ and ScanNet++ [52]² being subject to custom terms of use that require separate inspection, and to the Pixar Kitchen Set, which is distributed under Pixar’s non-commercial USD asset license.

B. Runtime Analysis

We report runtime statistics for 3D-Prog across select evaluation tasks in Tab. S3. Measurements were collected on an NVIDIA RTX 5090 GPU with 32 GB of memory.

C. User Study

User studies with four independent evaluators assessed 3D-Prog’s performance on object editing and compositional reasoning tasks. Evaluators scored each VLM-generated 3D asset from 0 to 5, following five predefined task-specific success criteria. For each instance, the evaluators were provided with the user prompt and the 3D visualization of the generated asset in a 3D USD visualizer. For object editing, the input 3D asset was also provided to check for geometry preservation. The final task performance was the average score from the four evaluators.

Compositional reasoning criteria cover semantics, orientation, color/material accuracy, structure, and composition

¹Available at https://kaldir.vc.in.tum.de/scannet/ScanNet_TOS.pdf (last accessed: 27 August 2026).

²Available at <https://kaldir.vc.in.tum.de/scannetpp/static/scannetpp-terms-of-use.pdf> (last accessed: 27 August 2026).

(detailed in Tab. S4). Object editing criteria include geometry preservation, material accuracy, realism of physical properties, scope accuracy, and visual realism (detailed in Tab. S5).

Analysis of the evaluators’ scores shows that in compositional reasoning, the baseline suffered from mesh intersections, incorrect orientations, and indistinguishable semantics, particularly when handling non-cubic primitives. Its success was limited to color and material attributes. In contrast, 3D-Prog achieved consistently high performance across all criteria. Regarding object editing, evaluators reported comparable visual quality between methods; however, the baseline consistently failed to execute physical property corrections.

D. TAF Analysis

In this section, we conduct a thorough analysis of the TAF component, with Sec. D.1 providing an inspection of the TAF feedback mechanism and Sec. D.2 giving an in-depth walkthrough of an example rollout of 3D-Prog on the semantic segmentation task with a 3-step iterative refinement.

D.1. Structure of TAF Outputs

TAF is a central component of 3D-Prog, enabling it to (a) extract semantic knowledge from VLMs, (b) reason about task execution in an iterative closed loop, thereby improving geometric fidelity and performance with respect to the number of iterations, and (c) address open-ended tasks.

We conducted a manual inspection of the 3D-Prog logs to analyze the capabilities of the TAF. The typical format of TAF follows the structure below, where each component exhibits distinct capabilities.

- **Analysis:** As instructed in the prompt, the VLMs perform both visual and structural (geometric) analysis for the given tasks, largely independent of the task specifics. This includes (a) inferring the axis of the 3D asset, (b) visually estimating shapes, sizes, and semantically identifying the 3D assets, and (c) inspecting the bounds of the asset.
- **3D Programming:** Uses the results of the analysis to carry out highly task-specific operations. Several steps may be carried out to gather more details about the asset, including (a) geometric filtering to remove noise, (b) finding the region of interest and visually zooming in, (c) PCA of the asset to estimate the canonical pose and the axis of symmetry, and (d) clustering objects and analyzing the individual clusters.
- **Reasoning:** Based on the analysis and the 3D programming outputs, the model draws conclusions that guide the next steps of the iteration. This typically involves (a) finding flaws in the analysis and the 3D programming (e.g., DBSCAN clusters are too coarse); (b) providing candi-

Table S1. Summary of 3D programming tasks we demonstrate, spanning generation, understanding, and reasoning over 3D assets.

	Task	Description	Input Asset
Generation	Object Completion	Complete partial object geometry into complete shapes.	3D object synthetic [13, 44]
	Scan2Sim	Convert a real-world scan into a simplified simulation-ready asset.	3D object scan [17]
Understanding	Object Detection (Spatial)	Detect and localize objects with bounding boxes for perception or manipulation tasks.	3D scene scan/ synthetic [44, 52]
	Object Part Segmentation (Geometric)	Segment objects into parts or sub-components with semantic and/or instance labels.	3D object synthetic [13, 44]
	Object Editing (Visual)	Modify object textures and materials; adjust object pose or placement at scene level.	3D object synthetic [13, 44]
	Situated QA (Contextual)	Spatial reasoning from a situated perspective.	3D scene scan [34]
	Object Captioning (Semantic-Spatial)	Given a 3D scene scan and a 3D bounding box of an object within it, caption the object.	3D scene scan [1]
Reasoning	Compositional Reasoning	Build a coherent scene from a given set of objects by reasoning about spatial and semantic relationships.	Text prompt (opt. 3D objects)
	Articulation Prediction	Given a 3D scene and a point marking a movable component, the model predicts the motion axis of the marked part.	3D scene scan [17]

Table S2. Licenses and terms of use for datasets employed in our study.

Dataset	License / Terms of Use
SQA3D [34]	Apache 2.0
Nr3D [1]	MIT
ScanNet [12]	ScanNet Terms of Use
ScanNet++ [52]	ScanNet++ Terms of Use
Articulate3D [17]	CC BY 4.0
Objaverse-XL [13]	ODC-By 1.0 (per-object licenses vary)
Pixar Kitchen Set [44]	Pixar non-commercial asset license

Table S3. **Runtime analysis.** Total time in seconds. Per-iteration time includes VLM inference ($\sim 60\%$), code execution ($\sim 25\%$), and rendering ($\sim 15\%$). Tasks requiring finer geometric reasoning (segmentation) use more iterations than semantic tasks (QA, captioning).

Task	Avg. Iter.	Total (s)	Per Iter. (s)
Object Completion	12.7	265	21
Object Detection	12.0	287	24
Semantic Seg.	19.2	446	23
Object Captioning	7.4	216	29
Situated QA	9.8	195	20
Articulation	7.3	125	17
Average	11.4	256	22

date solutions to resolve these flaws (reducing the DB-SCAN ϵ for fine-grained clusters).

- **Visual Validation:** The final stage involves visually inspecting the output to verify its correctness and validating the geometry and semantics of the asset.

Table S4. Success criteria for compositional reasoning.

Criterion	Description
Semantics	The overall compositional structure and scene semantics must exactly match task instructions from multi-angle inspection.
Orientation & Alignment	Objects with implied directionality must be oriented realistically and consistently with natural placement or user requirements.
Color & Material	Objects must have the correct specified colors.
Structure	No unrealistic overlapping or intersection of objects unless explicitly required by the task.
Composition	Objects must maintain correct compositional relationships (e.g., stacked/contained objects must not float or clip).

Table S5. Success criteria for object editing.

Criterion	Description
Geometry Preservation	The object’s underlying geometry must remain exactly the same as the original (incl. proportions, rotations, translations, etc.).
Material Accuracy	The edited material must visually match the intended type.
Physical Properties	Roughness, density, metallic, specular properties, etc. must be correct.
Scope Accuracy	The edit is correctly implemented over the specified scope, regardless of whether that scope is the entire object or a localized part.
Visual Realism	The object must behave visually in a way consistent with the intended material (e.g., proper metallic reflectivity, ceramic glossiness). Highlights and reflections must behave correctly.

D.2. Iterative Refinement Case Study: Semantic Segmentation

We perform a case study investigating the TAF visualizations across different 3D-Prog iterations. For the case study, we consider a semantic segmentation task on a synthetic tea kettle instance provided as a point cloud, as shown in Fig. S1.

Iteration 1: Initial Analysis and Hypothesis Formulation. The first visual program checked the asset’s up-axis and traversed the stage hierarchy, establishing that the asset contains a single mesh. Subsequently, the VLM calculated the bounding box and formulated an initial hypothesis that the semantic parts of the kettle are *Main Body*, *Lid*, *Spout*, *Handle*, and *Kettle Base*. The generated TAF was a 6-panel plot showing front, right, and top views, two 3D views, and a height distribution histogram.

The generated multi-view point-cloud analysis visualization was utilized to confirm the initial hypothesis regarding the presence of the five target components and to establish the basis for segmentation using Z (height) and R (radial distance from the XY centroid) as primary features.

Iteration 2: Segmentation and Validation. The initial segmentation algorithm employed geometric filtering for the *Base* and *Lid* (top 20% of Z -range). The middle section was separated into the *Body* (points within the 70th percentile radial distance) and an outer region.

The outer points were subjected to DBSCAN clustering to distinguish the *Spout* and *Handle* based on size and vertical spread. The second TAF, a 6-panel visualization with semantically colored points, was generated for visual inspection.

Iteration 3: Refinement with Enhanced Spatial Heuristics. The visual inspection of the TAF showed under-segmentation on the *Lid* and *Main Body* parts, as well as over-segmentation on the *Handle*. The VLM refined the initial segmentation algorithm with the following adjustments:

1. **Threshold Adjustment:** Height thresholds for the *Base* and *Lid* were slightly narrowed (12% and 18%, respectively), and the radial threshold was increased to the 75th percentile to ensure the *Handle* and *Spout* were fully contained within the outer region.
2. **Refined Clustering:** DBSCAN parameters were adjusted to $\text{eps}=2.5$ and $\text{min_samples}=3$ for improved density sensitivity.
3. **Geometric Differentiation Heuristics:** The critical refinement was the application of post-DBSCAN geometric heuristics to definitively classify each resulting cluster as either the *Spout* or the *Handle*. The *Handle* was identified by its arc shape and vertical dominance, whereas the

Spout was identified by its relatively compact and linear protrusion.

The legend of the analyzed TAF from iteration 2 also reveals a semantic labeling error—the segment labels were distributed incorrectly (e.g., the *Handle* was called “Lid”, the *Spout* was labeled as “Handle”). This inconsistency was obvious in the legend of the plot, but the VLM failed to attend to this non-geometric metadata and did not attempt to correct it, retaining the segment’s misclassification.

The third TAF is another panel with different views of the semantically colored point cloud, which confirmed the successful geometric segmentation. Crucially, while the geometric output was correct, the VLM did not self-correct the initial semantic labeling error, and the *components remained incorrectly labeled*. Notably, this failure to self-correct the labeling is observed very rarely and may be linked to the complexity of the task and to the VLM hallucinations.

E. Error Analysis

For each of the runs, we manually inspect the breakdown of the error modes of 3D-Prog. This analysis is essential to understand both the strengths of our method and its current limitations. We categorize the primary error modes as follows:

- **Incorrect Program:** The most common failure mode is the method producing erroneous code or invalid USD asset formats. This error mode is more common in the baseline system, often resulting in outputs that cannot be rendered. Iterative feedback significantly reduces these issues in 3D-Prog.
- **Insufficient Geometric Fidelity:** The final output sometimes lacks fine-grained geometric details. In some cases, 3D-Prog oversimplifies the geometry, behaving “lazily” and failing to capture intricate structures. The independent judge helps reduce this issue by enforcing stronger critical feedback.
- **Trivial Mistakes by Hallucination:** These occur when there is a mismatch between the model’s analysis and its final output. In a few runs, 3D-Prog correctly reasons about the scene but fails to transfer the same findings into the generated output, mainly due to hallucinations in the VLMs, as illustrated by the example in Sec. D.2.

F. Qualitative Results

We present additional qualitative results in Fig. S2 to further demonstrate the capabilities of our proposed framework. In object editing, 3D-Prog successfully modifies both the appearance and physical characteristics. For object detection, we demonstrate the identification of chairs within a cluttered environment, effectively handling incomplete scans and multiple true positives. Finally, we highlight 3D-Prog’s

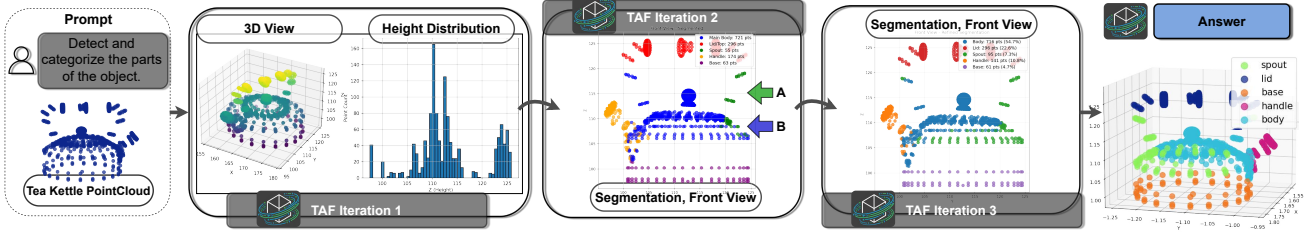


Figure S1. **TAF Visualizations Across Iterations.** The first iteration focuses on initial analysis for object understanding and label set prediction. In the second iteration, the TAF visualizes the initial segmentation of the tea kettle. We observe that the first segmentation exhibits over-segmentation (A), as well as under-segmentation issues (B). Those issues are detected by the VLM through the TAF and refined in the third iteration.

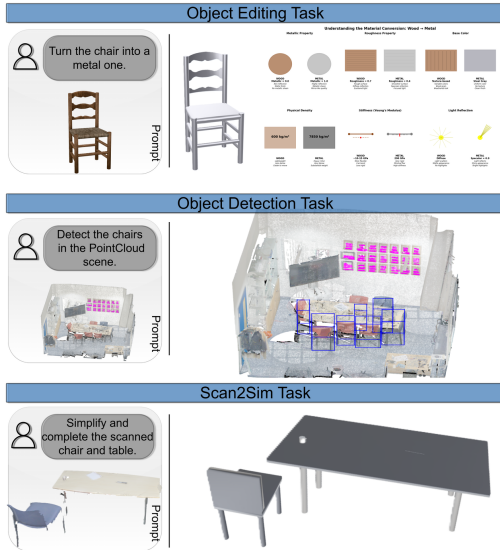


Figure S2. **Qualitative Results on 3D Object Editing, 3D Object Detection, and Scan2Sim Tasks.** For object editing, 3D-Prog generates a new USD asset with edited appearance and the corresponding physical properties. For object detection, the output is a list of bounding boxes in textual format, and in Scan2Sim the generated asset is the USD file of the simplified object.

ability to simultaneously perform scan simplification and completion to produce simulation-ready assets across diverse object classes and sizes.

F.1. 3D-Prog for 3D Programming in the Wild

We further validate 3D-Prog on self-captured real images taken in the wild with a phone camera. The first apple is reconstructed using SAM 3D [10]. Then we run 3D-Prog for various tasks, namely a) Generation and scene composition: to generate and place the table. The table needs to be of reasonable dimensions and texture. b) Object editing: to generate another apple and make its texture metallic. We show results in Fig. S3, where the table and the second apple are generated by 3D-Prog.

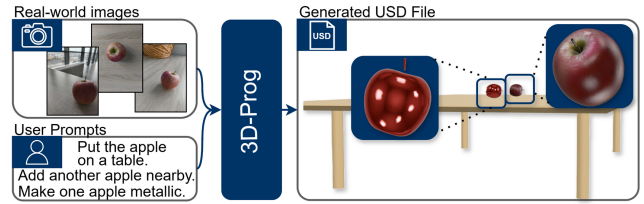


Figure S3. Zero-shot 3D-Prog results using self-captured RGB images.

G. Design Choices

In this section, we detail the specific implementation strategies. First, we elaborate on the 3D-Prog algorithm (Sec. G.1), and in Sec. G.2 we provide the full prompts utilized within the framework.

G.1. 3D-Prog Algorithm

In this section, we provide the complete algorithmic framework. Specifically, Algorithm 1 details the Visual Reasoning for 3D Programming process. This algorithm formalizes how the system repeatedly uses multi-view visual feedback to:

1. Analyze the target or generated geometry.
2. Generate Universal Scene Description (USD) code for model manipulation.
3. Execute the code to produce a new 3D model.
4. Compare the output against a reference.
5. Generate corrective feedback for the VLM.
6. Evaluate the final result for task compliance.

This iterative structure ensures that the VLM is guided by continuous visual and textual feedback until the generated USD model $\mathcal{M}$ satisfies the task requirements T .

G.2. Full Prompts for 3D-Prog

In this section, we detail the prompt structure for 3D-Prog. 3D-Prog includes iterative prompting of the 2D VLM acting as a “programmer” and the judge module.

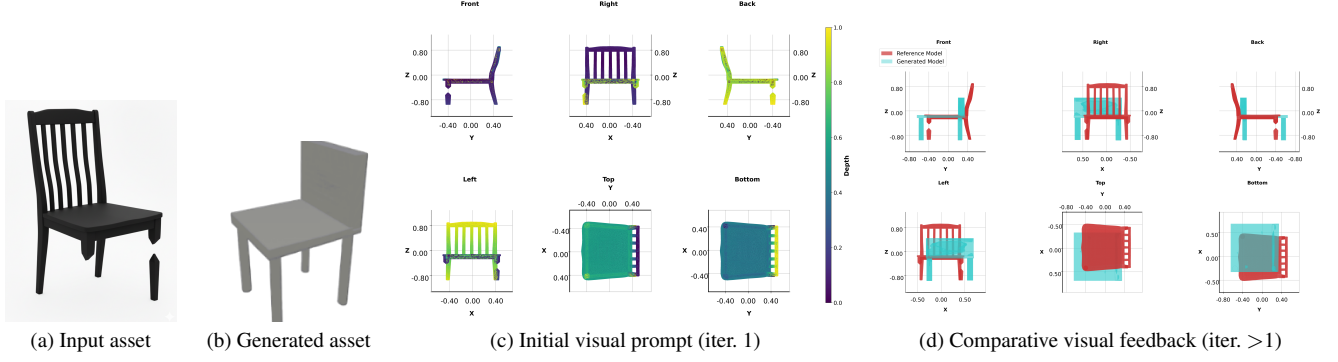


Figure S4. **Visual prompt for 3D-Prog.** (a) Input asset and (b) the asset generated in iteration 1. (c) The initial visual prompt after CCF: six orthographic views with depth-mapping. (d) The comparative visual feedback used from the second iteration on: six orthographic views overlaying the current generated asset (cyan) on the reference asset (red, when available). The complete visual prompt for iterations >1 additionally contains the TAF image, which varies with the task and iteration.

Algorithm 1 Visual Reasoning for 3D Programming

Require: Reference model $\mathcal{R}$ (optional), Task specification T , VLM $\mathcal{V}$
Ensure: Generated USD model $\mathcal{M}$ satisfying task requirements

```

1:  $\mathcal{I} \leftarrow \text{RenderMultiView}(\mathcal{R})$   $\triangleright$  6 orthographic views;  $\emptyset$  if no  $\mathcal{R}$ 
2:  $iteration \leftarrow 0$ ;  $feedback \leftarrow \emptyset$ 
3: repeat
4:    $iteration \leftarrow iteration + 1$ 
5:    $\mathcal{A} \leftarrow \mathcal{V}.\text{AnalyzeGeometry}(\mathcal{I}, T, feedback)$   $\triangleright$  Extract geometric features
6:    $code \leftarrow \mathcal{V}.\text{GenerateUSD}(\mathcal{A})$   $\triangleright$  Generate manipulation code
7:    $\mathcal{M} \leftarrow \text{Execute}(code)$   $\triangleright$  Create/modify USD geometry
8:    $\mathcal{I}_{gen} \leftarrow \text{RenderMultiView}(\mathcal{M})$ ;  $\mathcal{C} \leftarrow \mathcal{I}_{gen}$ 
9:   if  $\mathcal{R}$  exists then
10:     $\mathcal{C} \leftarrow \text{CompareVisual}(\mathcal{M}, \mathcal{R})$   $\triangleright$  Overlay comparison
11:     $feedback \leftarrow \mathcal{V}.\text{AnalyzeDiscrepancy}(\mathcal{C})$ 
12:   else
13:     $feedback \leftarrow \mathcal{V}.\text{EvaluateQuality}(\mathcal{I}_{gen})$ 
14:   end if
15:    $approved \leftarrow \text{Judge.Evaluate}(\mathcal{M}, T, \mathcal{C})$ 
16: until  $approved$  or  $iteration \geq \text{MAX\_ITERATIONS}$ 
17: return  $\mathcal{M}$ 

```

2D VLM Prompts. Visual Prompt. The initial visual prompt (Fig. S4c) is a composite image that consolidates six orthographic views of the asset.

For every subsequent iteration, the visual prompt be-

comes a two-part feedback mechanism designed to guide the VLM toward correction and refinement:

1. **Comparative Visual Feedback** (Fig. S4d): This first image provides a standardized direct comparison through six orthographic views. It plots the currently generated 3D asset and the reference asset (when available) following CCF. This visual overlay enables the VLM to readily identify and localize geometric discrepancies between its output and the target shape.
2. The second image is the TAF, which is dynamic and task-dependent. This feedback image contains targeted visual or textual information relevant to the specific error or refinement required in that iteration.

Textual Prompt. The 2D VLM receives a structured textual prompt composed of three distinct components:

- **System prompt** (Prompt 1): Outlines the agent’s capabilities, accessible libraries, I/O methods, and behavior. Defines CCF rules for 3D geometry understanding.
- **USD-core modeling rules** (Prompt 2): Provides specific instructions on using the USD-Core Python library, including syntax guidelines and a list of common errors that frequently lead to compilation failures or invalid USD file generation.
- **User prompt:** The task given by the user.

Beginning with the second iteration, the textual prompt is dynamically augmented by appending judge feedback (Prompt 4) on the previous iteration’s result to guide the agent toward convergence on the correct solution.

Prompt 1: System Prompt

Role. You are an AI assistant that generates and executes Python code for USD 3D modeling.

Communication Style

- Provide detailed descriptions of observations, planning, and execution.

- Clearly explain reasoning and decision-making steps.
- Describe image content with specific visual detail.
- Narrate the problem-solving process step-by-step.
- Balance tool usage with explanatory text.

Important Guidelines

- After executing code, analyze the result and fix any errors before retrying.
- Request clarification using `request_user_feedback` when necessary.
- Validate results before calling `task_complete`.
- Ask the user for direction when the next step is unclear.
- When analyzing images, describe visual details and use them to guide code generation.

File Paths

- Always use the provided `OUTPUT_DIR` variable for file outputs.
- Do not hardcode file paths or redefine `OUTPUT_DIR`.
- Save files using `os.path.join(OUTPUT_DIR, filename)`.
- The directory is automatically generated in a timestamped folder under `output/`.
- All USD files must be saved to `OUTPUT_DIR` for automatic visualization.

Image and Visualization Workflow

- After analyzing images, reference those observations throughout the task.
- Generated USD files are automatically visualized by the system.
- Wait for automatic visualizations rather than requesting them again.
- Images, plots, or diagrams may be generated using libraries such as `matplotlib` or `PIL`.
- Saved images are automatically displayed for further analysis.
- Iteratively refine visualizations based on observed results.

Iterative Workflow

Analyze → Visualize → Execute → Visual Feedback → Improve

USD files saved in `OUTPUT_DIR` are automatically visualized from multiple angles, enabling iterative refinement of model geometry.

Available Python Packages. `numpy`, `pandas`, `matplotlib`, `scipy`, `PIL` (Pillow), `math`, `random`, `time`, `os`, `sys`, `pathlib`, `uuid`, `string`, `re`, `json`, `OpenEXR`, `Imath`, `datetime`, `pxr` (Pixar USD)

Prompt 2: USD-Core Modeling Rules

USD-Core Details

Reference Matching

- Bounding boxes must **not** be used as simplified models.
- Objects must be modeled using their real components (e.g., chair legs, seat, backrest).
- Each visible component should be represented with separate geometry.
- Geometry must match the reference object's proportions and structure.
- Do not introduce additional features or simplifications.

Coordinate System

- Always set the stage to Z-up:

```
UsdGeom.SetStageUpAxis(stage,
UsdGeom.Tokens.z).
```

- USD uses a right-handed coordinate system.
- Axis directions:
 - $+Z$ = Up
 - $+Y$ = Away from viewer
 - $+X$ = Right
- Ensure the coordinate system matches the reference model.

Bounding Box Computation

- Preferred method uses `UsdGeom.BBoxCache`.
- Extent-based bounding boxes can be used for simple analysis.
- Avoid functions such as `bbox.IsEmpty()` or `bbox.GetBox()`.
- Bounding boxes may differ from visual geometry; always verify visually.

Primitive Selection

- `UsdGeom.Cube`: axis-aligned rectangular shapes only.
- `UsdGeom.Cylinder`: axis-aligned cylinders only.
- `UsdGeom.Sphere`: spherical primitives.
- `UsdGeom.Mesh`: required for rotated, tilted, or complex geometry.

Rotation Constraints

- Rotated cubes or cylinders are not allowed.
- Any rotated geometry must be represented using `UsdGeom.Mesh`.
- Rotation transforms should follow the order:

Translate → Scale → Rotate

- Incorrect transform ordering can distort geometry.

Dimension Analysis

- Extract component measurements from the reference model.
- Record dimensions for each part (e.g., leg height, seat width).
- Use measured ratios to reproduce correct proportions.
- Visual guessing of dimensions should be avoided.

Output Directory Usage

- Never hardcode output paths.
- Use the provided `OUTPUT_DIR` variable.
- Create files using `os.path.join(OUTPUT_DIR, filename)`.
- All generated USD and image files must be saved in this directory.

Rotation Operations

- Use axis rotations when objects appear tilted or diagonal.
- Example rotations include:
 - `AddRotateXOp()` for pitch
 - `AddRotateYOp()` for yaw
 - `AddRotateZOp()` for roll
- Multi-axis rotations may be used for complex orientations.

Alignment Validation

- Generated models must align with reference objects in all six views: Front, Back, Left, Right, Top, and Bottom.
- Models must occupy the same 3D space as the reference.
- Scale and orientation must match within a 5% tolerance.
- Any misalignment must be corrected before completing the task.

Judge Prompt and Feedback. The visual input provided to the Judge module is identical to the iterative visual prompt used by the 2D VLM for iterations > 1 . It consists of two images: (1) Comparative Visual Feedback (Fig. S4d)

showing six orthographic views comparing the generated asset (cyan) and the reference asset (red), and (2) the TAF. The judge’s textual instructions (Prompt 3) specify the evaluation criteria, rules, and response format, and Prompt 4 gives the exact format of its feedback.

Prompt 3: Judge Prompt

Independent Task Completion Evaluation

Role

- Act as an independent judge evaluating whether a task has been completed successfully.
- Maintain objectivity and avoid bias toward approving completion.

Inputs

- **Original Task:** {task_description}
- **Agent Output:** {agent_output}

Evaluation Criteria

Task Requirements Fulfillment

- Verify that all aspects of the original task are addressed.
- Identify any missing components or incomplete implementations.

Quality Assessment

- Determine whether the produced work meets acceptable quality standards.
- Identify inconsistencies or obvious mistakes.

Code Quality (Programming Tasks)

- Verify that the code is functional and free of errors.
- Evaluate adherence to best practices.
- Detect runtime or logical errors.

Evaluation Rules

- Be highly critical and act as the quality gatekeeper.
- If comparison images are missing for USD tasks, automatically reject completion.
- Visual evidence overrides agent claims.
- If generated and reference models do not visually match, mark the task as failed.
- Default to rejecting completion unless strong evidence supports success.
- Statements such as “perfect alignment” must be verified visually.
- Provide specific and detailed feedback describing observed issues.
- For all 3D/USD tasks, comparison images are mandatory for approval.

Response Format

Provide the evaluation as JSON:

```
{
  "is_complete": true/false,
  "confidence": 0.0-1.0,
  "feedback": "detailed explanation",
  "errors": ["list", "of", "specific", "errors"],
  "suggestions": ["list", "of", "improvements"]
}
```

Prompt 4: Judge Feedback Format

```
{
  "is_complete": true/false,
  "confidence": 0.0-1.0,
  "feedback": "detailed explanation of
    ↳ your evaluation",
  "errors": ["list", "of", "specific",
    ↳ "errors", "found"],
  "suggestions": ["list", "of",
    ↳ "improvement", "suggestions"]
}
```